

UNIVERSITÉ PARIS CITÉ
UFR DE MATHÉMATIQUES ET INFORMATIQUE



Web Project - EventHub

M1 INFORMATIQUE

Année 2025 / 2026

Nic XU – 22235057

Amel KACI - 22200653

Introduction

EventHub is a full-stack Single Page Application (SPA) designed for event management. It allows authenticated users to register to event. Users with admin privilege can create and manage events as well as participants, through a clean, role-driven interface. The system enforces access control at every layer: unauthenticated users are redirected to the login page, viewers have read-only access, administrators enjoy full CRUD access across all entities.

Overview

Layer	Technology	Role
Backend	Django 4.x + Django REST Framework	REST API, business logic, ORM, JWT auth
Auth	JWT	Access/refresh token generation and validation
Database	SQLite (dev) / PostgreSQL (prod)	Relational data persistence
Frontend	React 18 + React Router v6	SPA, routing, component tree
State	Context API + custom hooks	Auth state, API data fetching
Comparative study	Node.js	Alternative backend implementation for class comparison

Node Vs Django Comparison

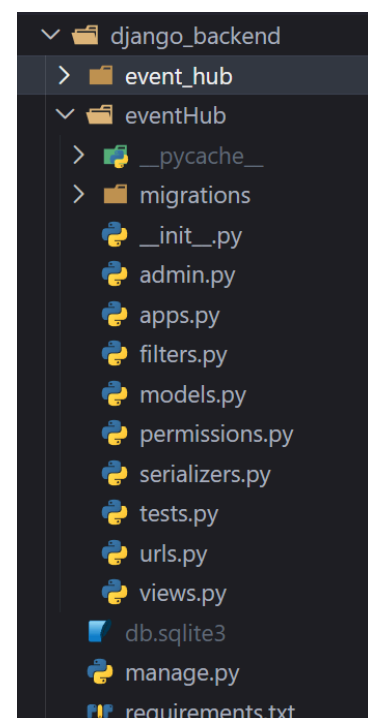
In scientific computing, we often see a similar pattern with C and Python. Python is usually preferred for experimentation, not only because it has a lower programming skill barrier, but also because its rich ecosystem of libraries makes it very versatile. However, when resources are scarce or performance becomes critical, scientists tend to move toward lower-level languages: they are harder to master but can be extremely efficient when well written. That is why modern flight control systems, for example, are typically implemented in C/C++ or similar languages.

For Django and Node.js, the situation is similar. Even though both are backend technologies, Django is Python-based and provides a more structured environment, with many features already available through integrated tools and libraries, making it easier to get started and deploy. Node.js, on the other hand, offers a freer and more flexible environment, but this freedom can be harder for beginners to handle.

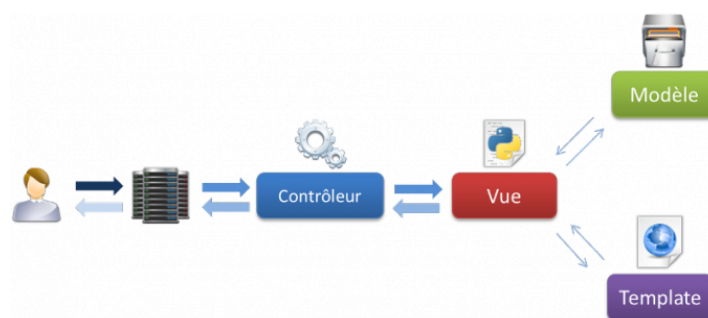
Architecture :

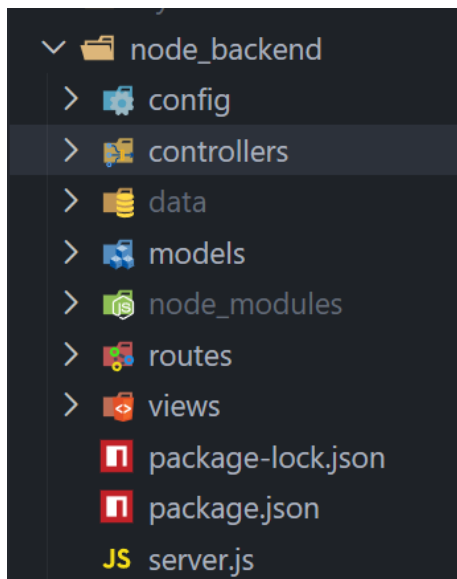
With Django, the project structure was already set up after running the python manage.py startapp command, the learning curve remained significant throughout the project, as it required getting used to the imposed environment and learning how to leverage the features already available in the framework and its libraries.

The architecture used by Django is MVT (Model, View, Template). It is similar to MVC (Model, View, Controller), where the Model defines the structure of the data in the database. The main difference lies in the role of the view: in MVC, the view is responsible for displaying information to the user, while requests are handled by the controller. In Django, the view acts as a request handler: it receives HTTP requests, decides what data to retrieve or process, and then renders a template by determining what to add to it.



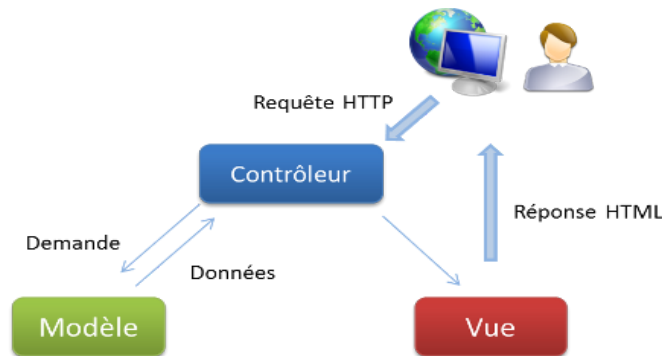
MVT Architecture :





With Node.js, it is up to us to choose which architecture to use. Since we had already worked on web projects using the MVC architecture during our bachelor's degree, we were able to apply the same pattern with Node. Once we decided on the architecture, the implementation was straightforward and fast to put in use, and the environment felt more minimalistic compared to Django. However, we did encounter several rounds of refactoring due to architectural changes. Node.js is also asynchronous and event-driven, which makes it well-suited for handling many concurrent requests efficiently.

WP7 : Express configuration driven -> we can see a config folder



In modern web projects, Node.js is often combined with a front-end library such as React, using JavaScript. This creates a full-stack JavaScript architecture, where the Node.js backend serves data through REST APIs and React renders the user interface dynamically in the browser. Django is usually used as a backend for data-driven applications, while the front-end can be built separately with React (or another framework), communicating with Django via JSON APIs. This keeps the back-end and front-end more independent but still allows a clear separation of concerns.

Complexity :

Django is “batteries included”. It enforces a structured architecture (MVT), and many tools are already built-in: an ORM, an admin panel, authentication, form handling, and basic security features. This reduces the number of decisions the developer has to make at the beginning, which speeds up development. However, the learning curve is moderate, because the developer must understand several concepts (models, views, URLs, apps, migrations) and adapt to the framework's conventions.

Node.js, often used with Express, follows a minimalist and unopinionated approach. Express itself is very lightweight and provides only basic routing and middleware. The developer must choose and integrate almost everything else: ORM/ODM, authentication, validation, logging, testing tools, etc. This makes the initial setup very simple and fast, but the complexity grows as the project evolves, because the architecture and tooling depend heavily on the developer's choices. In practice, this can lead to several refactoring phases when the architecture changes.

In short, Django offers more structure and built-in tools, which reduces early complexity but requires learning its conventions. Node.js / Express gives more freedom and starts very simply, but the overall complexity of the project depends strongly on the developer's architectural decisions.

Scalability :

According to WP7, Django is designed to be scalable & production-ready. Many large websites (such as Instagram, The Washington Post, ...) run on Django, which shows that it can handle high traffic when the architecture is well planned. For example for our use case, we could have swapped vercel with cloud services such as AWS which gives us lots of control and scalability as it handles high traffic according to WP10

Node.js, on the other hand, is built on an asynchronous architecture that can process requests as they arrive without waiting for a response. It can efficiently handle many concurrent connections with low memory usage, which makes it ideal for real-time applications.

In summary, Django is a strong choice for structured, data-heavy backends that may scale vertically with good optimization, while Node.js excels in highly concurrent, real-time, or microservice-oriented systems where horizontal scaling is a priority.[hostinger+2](#)

For this project, Django was assigned rather than chosen freely. However, it turned out to be suitable because the learning curve remained relatively constant throughout the development, and the application did not require much upfront work on the architecture. In contrast, Node.js would have forced us to make more architectural decisions early on, which can be difficult if we do not have a strong preference or prior experience with a specific pattern. Django's built-in structure therefore helped us focus on the core features without spending too much time designing the system from scratch.

Frontend

The frontend is a single-page application built with React 18 and React Router v6. It is structured around a central AuthContext, a generic useApi hook, and a set of page components mounted inside a protected shell layout.

Page Components Summary

Page	Route	Access	Description
Auth	/login	Public	JWT login form with error handling, switch to sign up
Home	/home	Auth	Upcoming registered events + quick actions
Dashboard	/dashboard	Admin	Stats, recent events, status breakdown chart
Events	/events	Auth	Filterable events table (status, location, search)
EventCreate	/admin/events/create	Admin	Event creation form
Event Detail	/events/:id	Auth	Full event info, participants grid, registration, admin actions
Participants	/participants	Admin	Searchable participant table with role badges
Profile	/profile	Auth	Info form, password change, delete account (for own account)
Profile	/profile/:id	Admin	Info form, password change, delete account (for any account)

Loading and Error State :

Every data-fetching operation is wrapped with explicit loading and error states:

- Loading: a CSS-animated spinner is shown while requests are in flight.
- API errors: displayed in a styled error box with the message extracted from the response body.
- Form errors: inline below the relevant form field, using field-keyed error dicts returned by DRF.
- Success feedback: green success boxes confirm save operations without disrupting the page flow.

API Integration

All communication between the React frontend and the Django backend happens over HTTP (HTTPS in production) using JSON. The frontend sends an HTTP POST to `/api/token/` with

the user's email and password; Django returns a short-lived access token and a refresh token. Every subsequent request includes the access token in the Authorization: Bearer <token>header.

Since Django is using a MVT architecture, let's dive into the models first. We have 3 tables :

- Participants : stores users' informations.
- Events : stores an event's informations
- Registrations : represents many-to-many relationship

The model ensures a many-to-many relationship between Participant and Event. It enforces a unique_together constraint (replaced by UniqueConstraint in modern Django) to prevent double registration.

For business validation, a clean() method validates capacity before registration is accepted. For example, the modalities of the rules for Events are like so :

- Both start_datetime/end_datetime required (or both null)
- the end_datetime has to be after the given date for
- check the event's capacity and places left to validate or not the registering.

For Each model, we have a dedicated ModelSerializer. The validate() method on each serializer instantiates a temporary model object and calls clean() on it, bridging Django's model validation into the DRF pipeline:

The ParticipantSerializer overrides create() and update() to hash passwords correctly via set_password() instead of storing raw strings.

Focusing on the views, as stated in the comparison between Node and Django, there is not a dedicated controller with Django, the [views.py](#) file the requests. The permission is often checked in 2 layers, the first one which is with the frontend by limiting the access to some components on the pages depending on the viewers.

```
isAdmin && (
    <li><NavLink to="/participants">Participants</NavLink></li>
)
```

However, we should not rely only on the view for permissions handling as it can be bypassed easily, the second and more robust one is to check directly in the backend side.

We use permission_classes[] with variables already implemented by Django, there are also custom made permissions in the file [permissions.py](#) which contains the following function :

- ReadOnly : returns only SAFE_METHODS
- IsOwnerPermission : reserved for the user
- IsAdminOrSelf : the registration can be seen only by the admin and the concerned user

We then proceed to test all the permissions handling with Postman to ensure that access to sensitive contents are limited

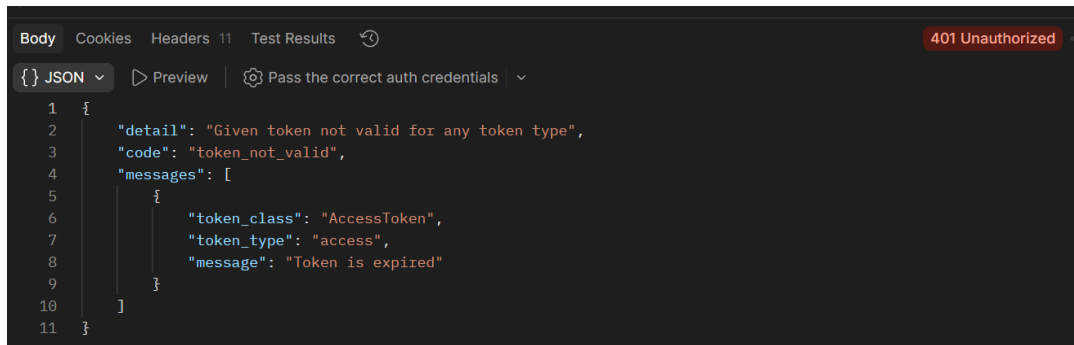
When the access token expires, the frontend calls `/api/token/refresh/` with the stored refresh token to obtain a new pair, keeping the session alive without forcing the user to log in again. All other routes (`/api/events/`, `/api/participants/`, `/api/registrations/`) are guarded by `IsAuthenticated` at the Django level, so even if a frontend route guard is bypassed, the API returns 401 Unauthorized.

```
class RegistrationViewSet(ModelViewSet):
    serializer_class = RegistrationSerializer
    permission_classes = [IsAdminUser | IsOwnerPermission]

    def get_queryset(self):
        if self.request.user.is_staff:
            queryset = Registration.objects.all()
        else:
            queryset = Registration.objects.filter(participant=self.request.user)

        event_id = self.request.query_params.get("event")
        if event_id:
            queryset = queryset.filter(event_id=event_id)

        return queryset
```



API Endpoints

Method	Endpoint	Role	Description
GET	<code>/api/events/</code>	All	List events (with filtering)
POST	<code>/api/events/</code>	Admin	Create event
GET	<code>/api/events/{id}/</code>	All	Event detail
PATCH	<code>/api/events/{id}/</code>	Admin	Update event
DELETE	<code>/api/events/{id}/</code>	Admin	Delete event
GET	<code>/api/participants/</code>	Admin	List all participants
GET	<code>/api/participants/me/</code>	All	Own profile
PATCH	<code>/api/participants/{id}/</code>	Owner/Admin	Update profile
DELETE	<code>/api/participants/{id}/</code>	Owner/Admin	Delete account
POST	<code>/api/registrations/</code>	All	Register for event
DELETE	<code>/api/registrations/{id}/</code>	Owner/Admin	Unregister
POST	<code>/api/token/</code>	Public	Obtain JWT pair
POST	<code>/api/token/refresh/</code>	Public	Refresh access token

Deployment

So up until now we have been able to create a full-stack single page application that works from end-to-end. We were not able to fully deploy our web app.

We prepared .env files that store API urls, secret keys...

For the backend we deployed using Render and is accessible with a fully functional admin interface. Our front-end is currently exposed to the public url

<https://myproject-theta-flame.vercel.app/login>

However it is not usable as it is as we were not able to solve a bug that prevents us from accessing the backend server.

Outcome

A Single Page Application written in ReactJs with a Django API. Works end-to-end locally and not yet deployed.